



MÓDULO I

Introducción a la Inteligencia Artificial y modelos fundacionales

Curso: Fundamentos de Inteligencia Artificial

Clase 2: Procesamiento de lenguaje natural y representación del conocimiento

Contenido



❑ Fundamentos de NLP y desafíos

- ¿Por qué el lenguaje es difícil para las máquinas? Ambigüedad, contexto, pragmática
- Pipeline clásico de NLP vs. enfoques end-to-end

❑ Técnicas clásicas

- Tokenización, POS tagging, parsing
- N-gramas, TF-IDF, modelos de Markov

❑ Representación del conocimiento

- Lógica proposicional y de predicados
- Frames, ontologías

❑ Word embeddings y representaciones distribucionales

- One-hot encoding a Word2Vec
- Embeddings estáticos a contextuales (BERT, GPT)
- RAG



Fundamentos de NLP y desafíos

¿Por qué el lenguaje es difícil para las máquinas?

Ambigüedad

Una palabra, frase u oración puede tener múltiples significados válidos dependiendo del contexto.

"Banco": ¿institución financiera o asiento?

"Vi a la chica con el telescopio": ¿quién tiene el telescopio?

Contexto

El significado depende de información previa, situación y conocimiento del mundo

"Él llegó tarde"

¿Quién es "él"? Requiere contexto previo

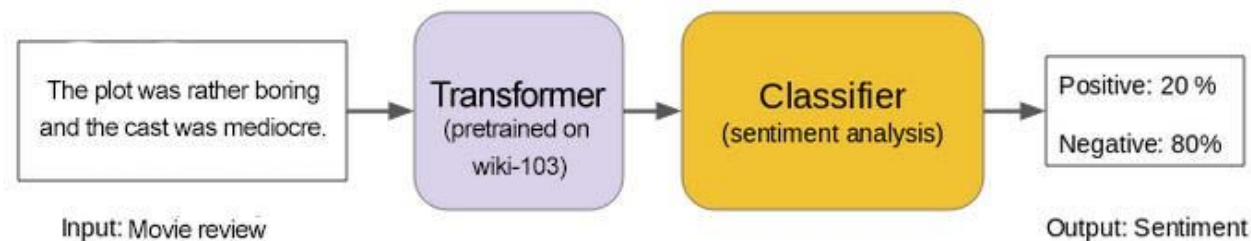
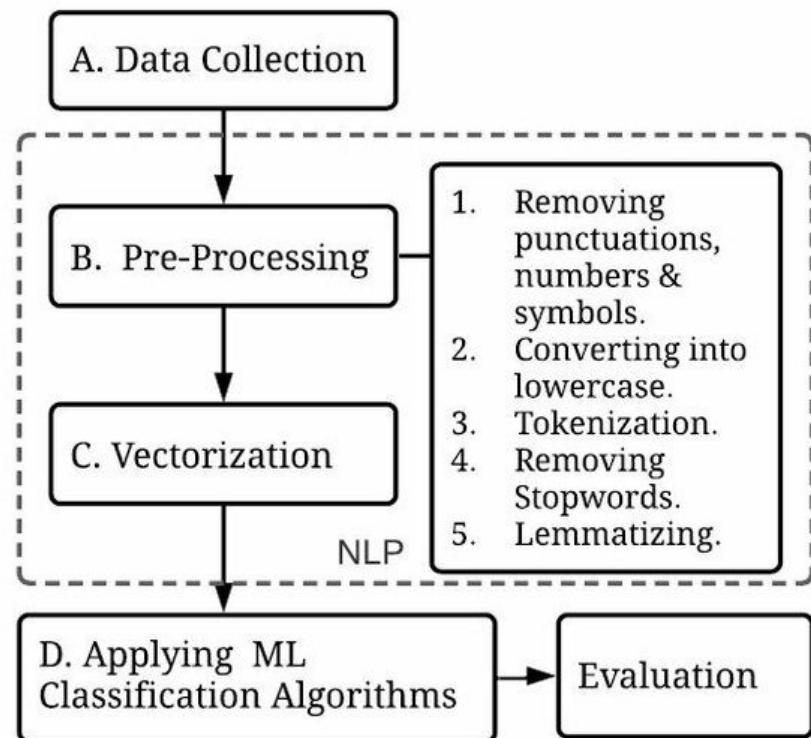
Pragmática

Lo que se quiere decir va más allá de lo que se dice literalmente

"¿Puedes pasarme la sal?"

No es pregunta sobre capacidad, es petición.

Pipeline clásico vs End-to-End



Ver código:
[colab](#)

Tokenización, POS Tagging y Parsing

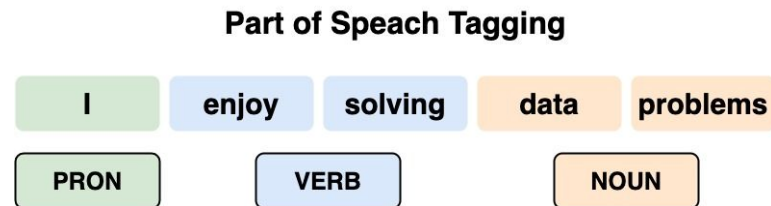
Tokenización

Proceso de dividir texto continuo en unidades discretas (tokens) como palabras, signos de puntuación o subpalabras.



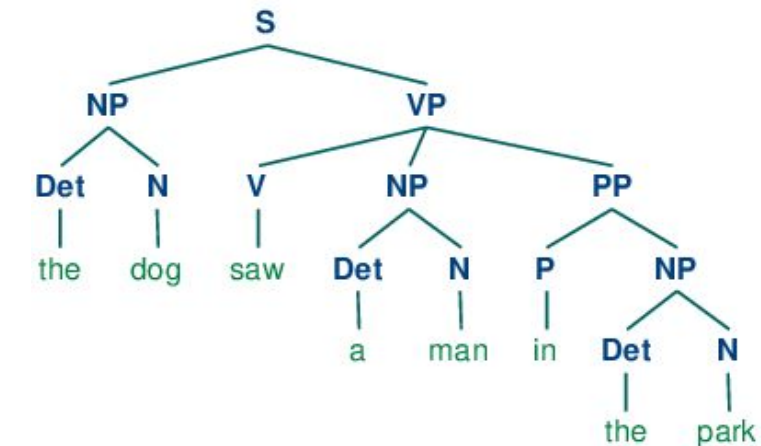
POS Tagging

Asignación de etiquetas gramaticales (Part-of-Speech) a cada palabra según su función sintáctica en la oración.



Parsing sintáctico

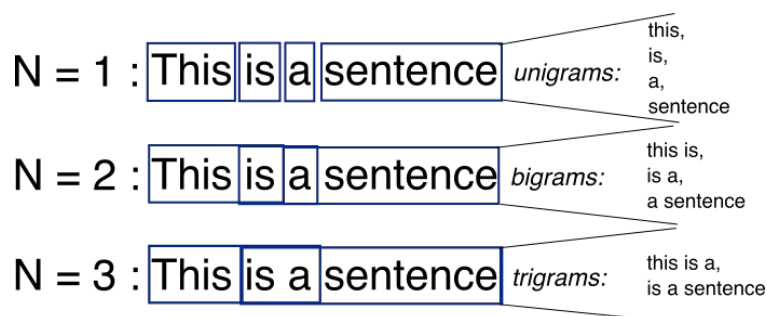
Análisis de la estructura gramatical para identificar relaciones sintácticas entre palabras y construir árboles de análisis.



N-gramas, TF-IDF y modelos de Markov

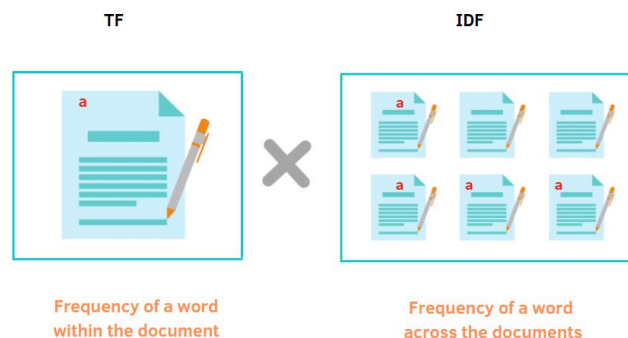
N-gramas

Secuencias contiguas de N elementos (palabras, caracteres) que capturan patrones locales y contexto limitado en el texto.



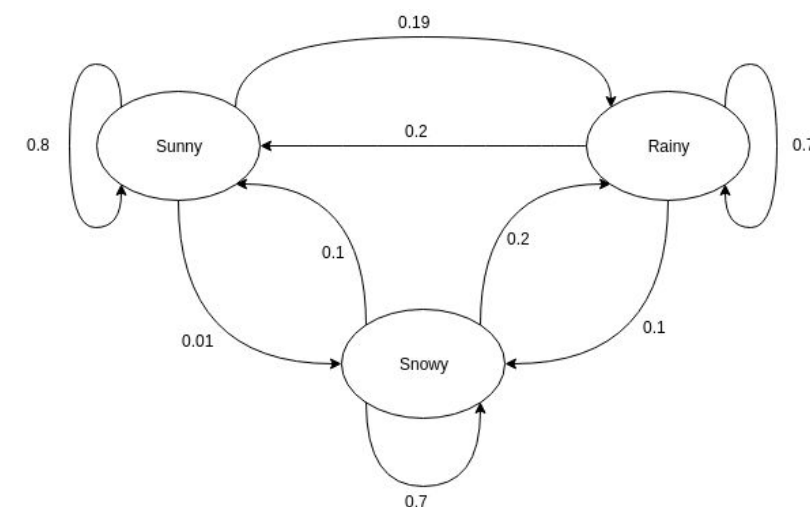
TF-IDF

Term Frequency - Inverse Document Frequency. Mide la importancia de una palabra en un documento relativo a una colección de documentos.



Modelos de Markov

Modelos probabilísticos donde la probabilidad del siguiente estado depende únicamente del estado actual (propiedad markoviana).





Break 10



Representación del conocimiento

Lógica proposicional y de predicados

Lógica proposicional

Sistema formal que trabaja con proposiciones (declaraciones que pueden ser verdaderas o falsas) y conectivos lógicos para expresar y razonar sobre el conocimiento.

P, Q, R : variables proposicionales

$\neg, \wedge, \vee, \rightarrow$: Operadores

Ejemplo de inferencia:

1. Llueve $\rightarrow$ QuedoEnCasa

2. Llueve

$\therefore$ QuedoEnCasa (Modus Ponens)

Lógica de predicados

Extensión de la lógica proposicional que permite representar relaciones, propiedades de objetos y cuantificación sobre dominios de elementos.

$P(x)$: Predicados con variables

$\forall x$: Cuantificador universal

$\exists x$: Cuantificador existencial

Ejemplo de inferencia:

1. $\forall x (Gato(x) \rightarrow Animal(x))$

2. Gato(garfield)

$\therefore$ Animal(garfield)

Frames y ontologías

Frames

Estructuras de datos que representan conocimiento sobre objetos, situaciones o conceptos mediante slots (atributos) y valores, con herencia y valores por defecto.

```
Frame: Persona
  nombre: [requerido]
  edad: [numero, >0]
  profesion: [string]
  saluda: [metodo]

Frame: Estudiante (hereda
Persona)
  universidad: [string]
  carrera: [string]
  estudia: [metodo]

Instancia: juan
  tipo: Estudiante
  nombre: "Juan Perez"
  edad: 20
  universidad: "UPC"
```

<https://deepsense.ai/resource/ontology-driven-knowledge-graph-for-graphrag/>

Ontologías

Especificaciones formales y explícitas de conceptualizaciones compartidas, que definen vocabularios, taxonomías y axiomas para un dominio específico.

```
owl:Thing
  Servivo
  Animal
    Mamifero
      tieneColumnaVertebral: true
      respiracion: pulmonar
    Ave
      tieneAlas: true
      puedeVolar: boolean
  Planta
    realizaFotosintesis: true

Axiomas:
  Mamifero  $\subseteq$  Animal
  Ave  $\subseteq$  Animal
  Animal  $\cap$  Planta =  $\emptyset$ 
```

One-Hot Encoding y Word2Vec

One-Hot Encoding

Representación binaria donde cada palabra se codifica como un vector con un solo 1 en la posición correspondiente al índice de la palabra en el vocabulario.

Vocabulario: ["gato", "perro", "come", "duerme"]

gato = [1, 0, 0, 0]

perro = [0, 1, 0, 0]

come = [0, 0, 1, 0]

duerme = [0, 0, 0, 1]

Frase: "gato come" = [[1,0,0,0], [0,0,1,0]]

Word2Vec (2013)

Técnica que aprende representaciones densas de palabras capturando significado semántico basado en contexto: "palabras similares aparecen en contextos similares".

gato = [0.2, -0.1, 0.7, 0.3]

perro = [0.1, -0.2, 0.8, 0.4]

come = [-0.5, 0.6, -0.1, 0.2]

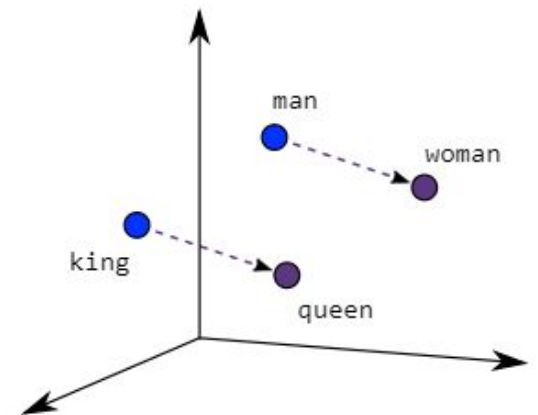
duerme = [-0.3, 0.4, -0.2, 0.1]

Similitudes coseno:

$\text{sim}(\text{gato}, \text{perro}) = 0.89$

$\text{sim}(\text{come}, \text{duerme}) = 0.78$

$\text{sim}(\text{gato}, \text{come}) = 0.12$



De embeddings estáticos a contextuales

Embeddings estáticos

Cada palabra tiene una representación vectorial fija, independiente del contexto en que aparezca. Word2Vec, GloVe, FastText.

"I went to the bank to deposit money"

bank = [0.2, -0.1, 0.7, ...] (institución financiera)

"I sat by the river bank to relax"

bank = [0.2, -0.1, 0.7, ...] (misma representación)

Embeddings contextuales

Cada palabra recibe una representación diferente según su contexto específico. Transformers analizan toda la secuencia simultáneamente.

"I went to the bank to deposit money"

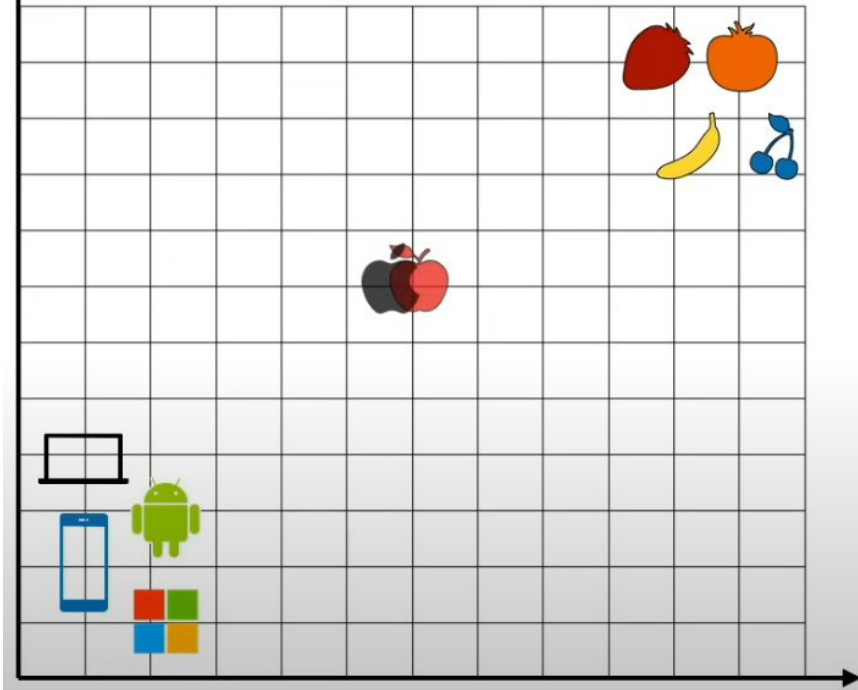
bank = [0.8, -0.2, 0.1, ...] (contexto: money, deposit)

"I sat by the river bank to relax"

bank = [-0.1, 0.6, 0.9, ...] (contexto: river, nature)

Embedding contextuales

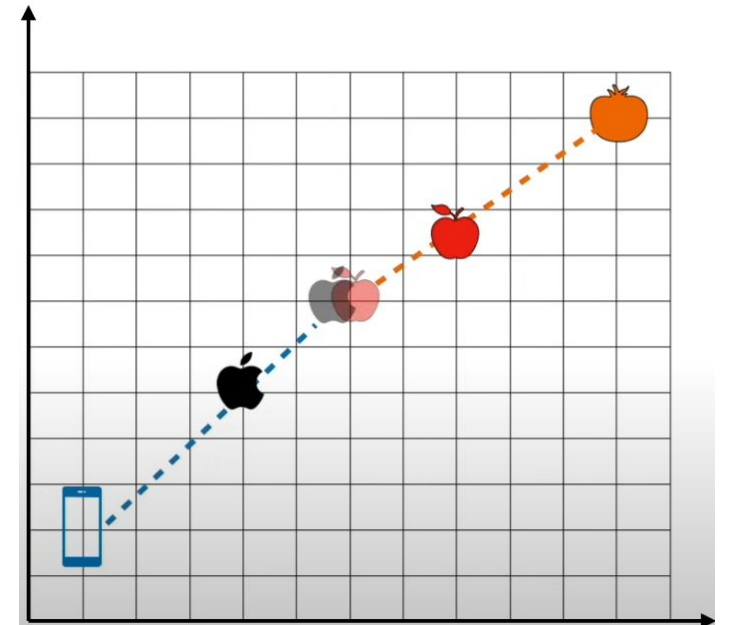
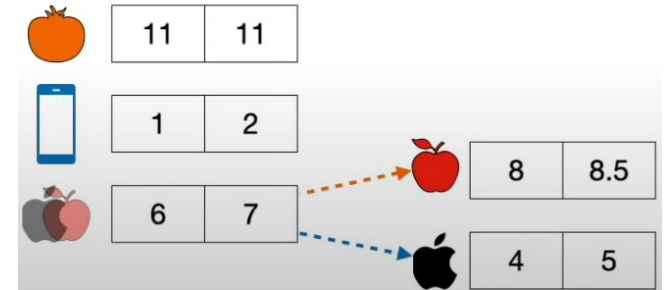
Embedding space



please buy an **apple** and an **orange**

apple unveiled the new **phone**

En la primera oración el **attention** daría más **peso** a **orange**, entendiendo que **apple** se refiere a la **fruta**. En la segunda oración se daría más peso a **unveiled** y **phone**, refiriéndose a la compañía **Apple**.



GPT, BERT, T5, CLIP

Modelo	Arquitectura	Principal diferencia	Usos
GPT	Transformer solo de decodificador	Enfoque autorregresivo para la generación de texto	Generación de texto, completado de texto
BERT	Transformer solo de codificador	Enfoque bidireccional para la comprensión profunda del contexto	Clasificación de texto, respuesta a preguntas, análisis de sentimiento
T5	Transformer de codificador-decodificador	Unificación de tareas bajo el enfoque de transformación de texto a texto	Traducción, resumen, clasificación, respuesta a preguntas
CLIP	Transformer multimodal (texto e imagen)	Mapeo de imágenes y texto en un espacio de embedding compartido	Búsqueda de imágenes, clasificación de imágenes, generación de descripciones a partir de imágenes

GPT

Generates human-like text based on input prompts.

Example:

Input: 'Describe a face' Output: 'A round face with bright blue eyes, a button nose, and full lips curved into a warm smile.'

BERT

Understands context in text for various NLP tasks.

Example:

Input: 'The woman had a [MASK] face.' Output: 'The woman had a beautiful face.'

T5

Transforms text input to text output for various tasks.

Example:

Input: 'Translate to French: She has a pretty face' Output: 'Elle a un joli visage'

CLIP

Connects text descriptions with relevant images.

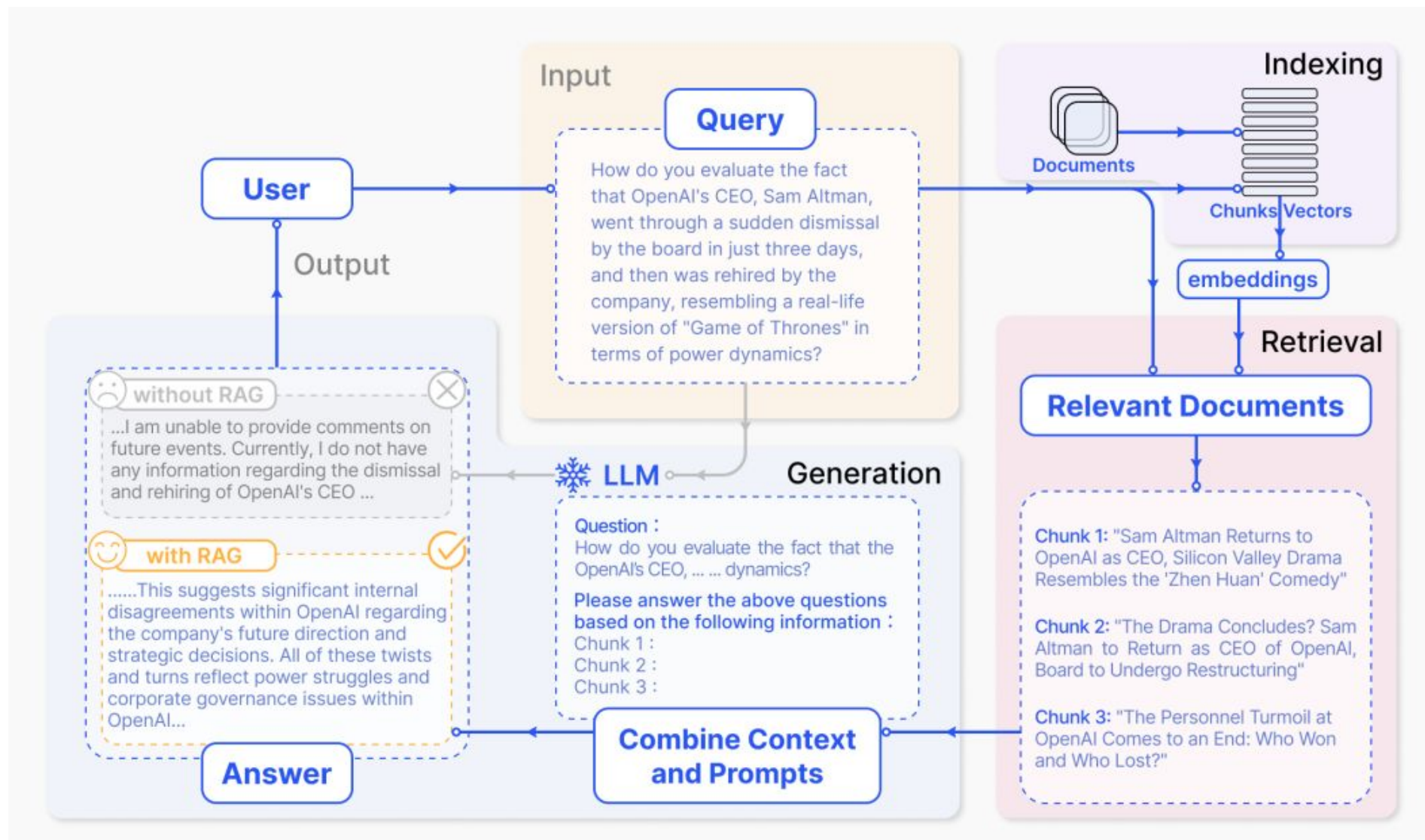
Example:

Input: Text: 'A smiling face', Images: [😊, 😐, 😞] Output: Selects 😊

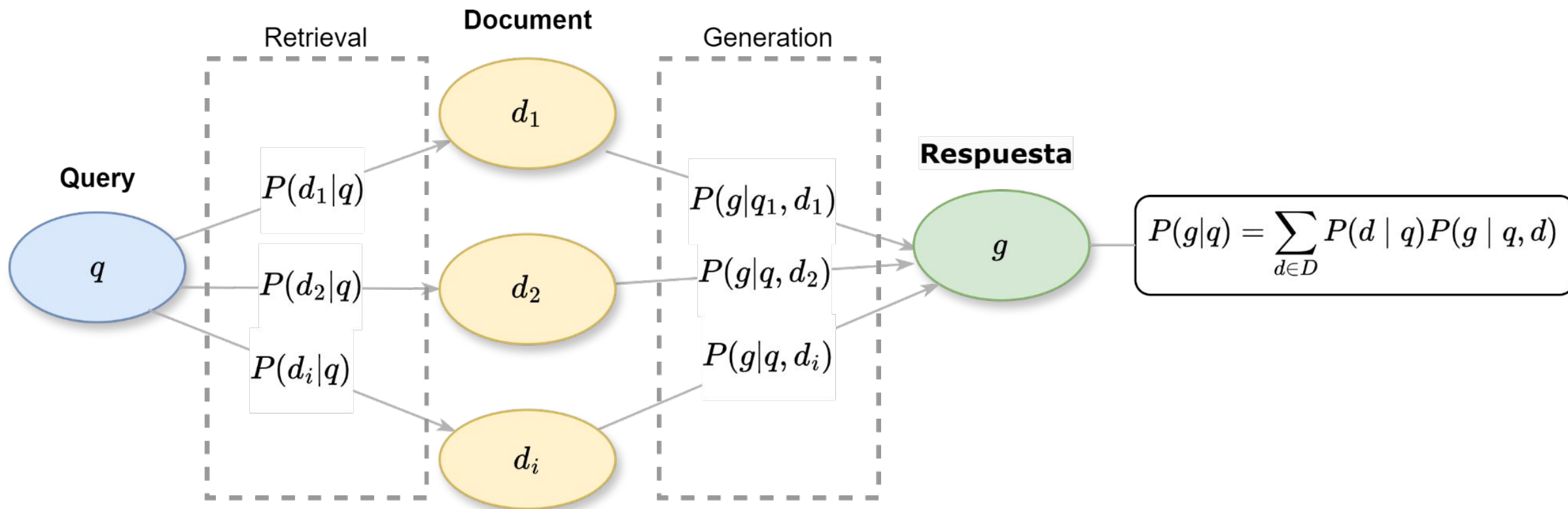


Break 10

Retrieval-Augmented Generation (RAG)

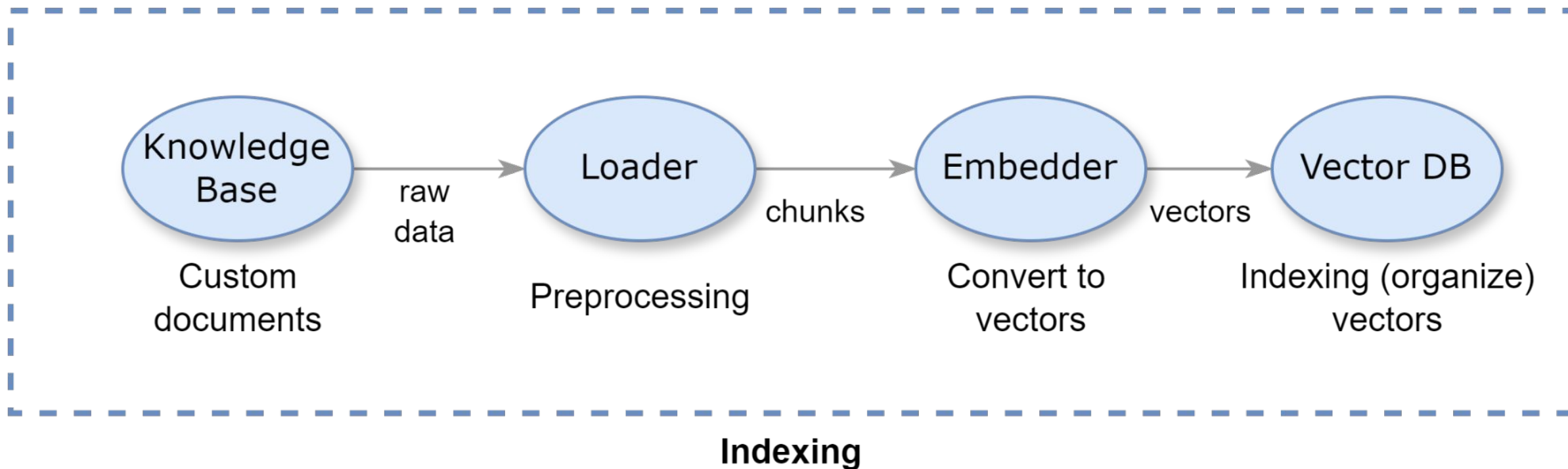


Formalmente



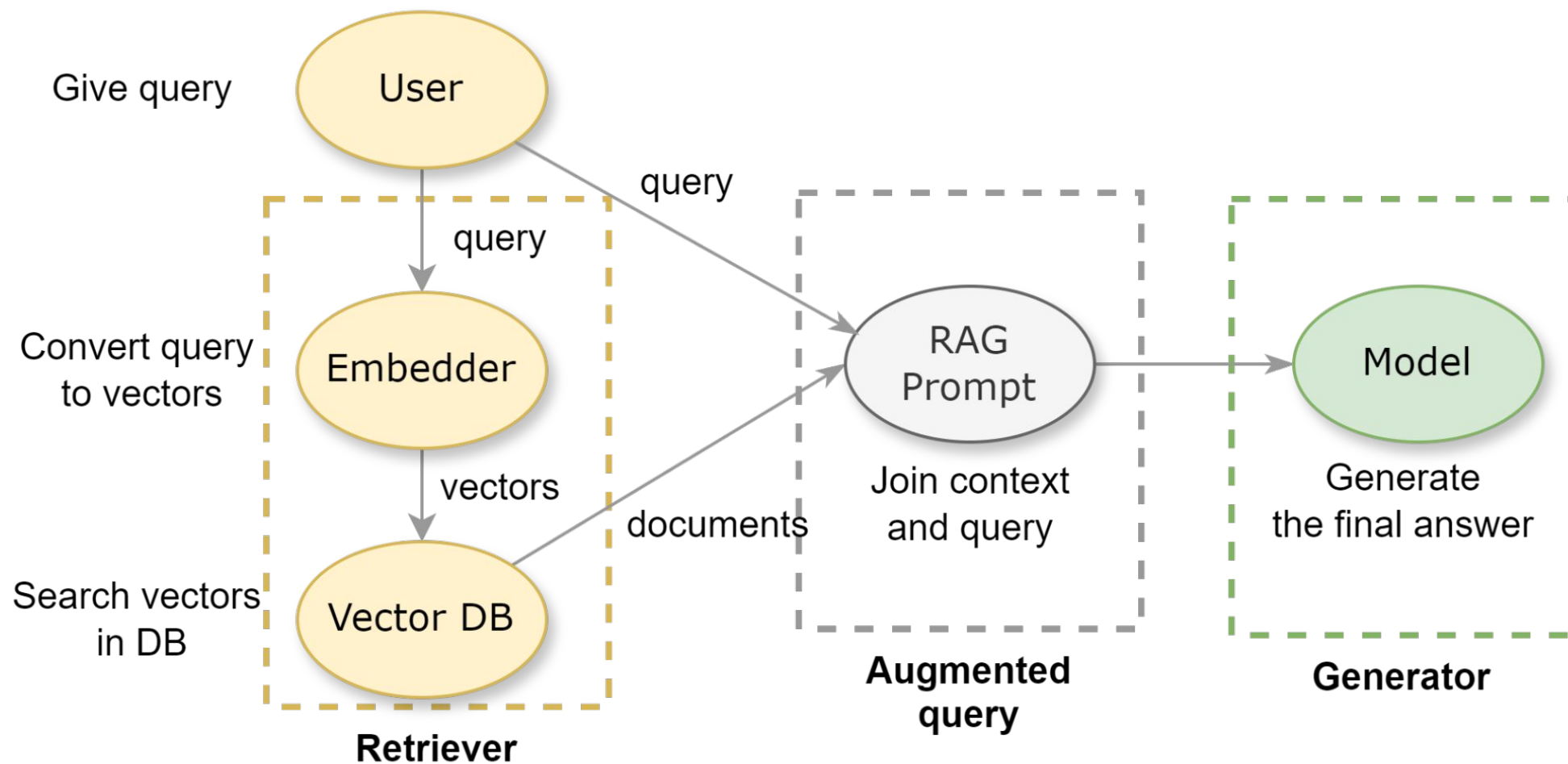
RAG se puede definir como la probabilidad condicional de generar una respuesta **g** dada una consulta **q** y un conjunto de documentos recuperados **D**.

RAG: indexing



El **indexing** significa organizar los vectores en una DB de tal manera que sean rápidamente encontrados.

RAG: inferencia



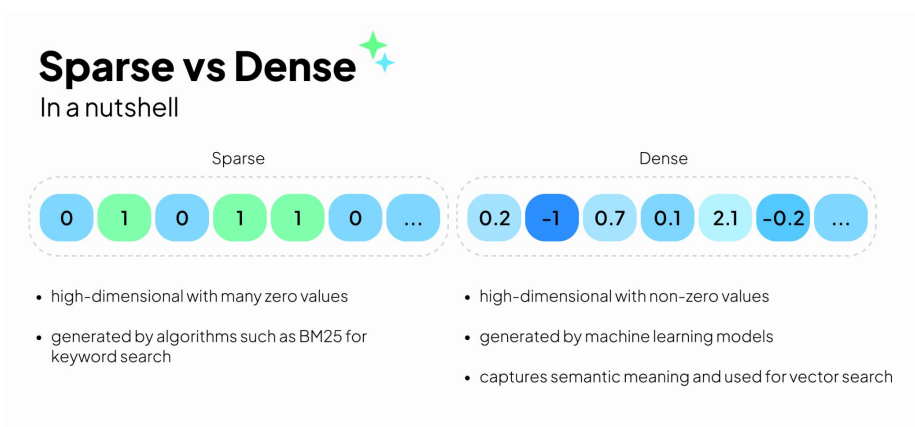
Módulo Retriever

El **retriever** se encarga de **obtener** un conjunto de **documentos** que sean **relevantes** para el **query**.

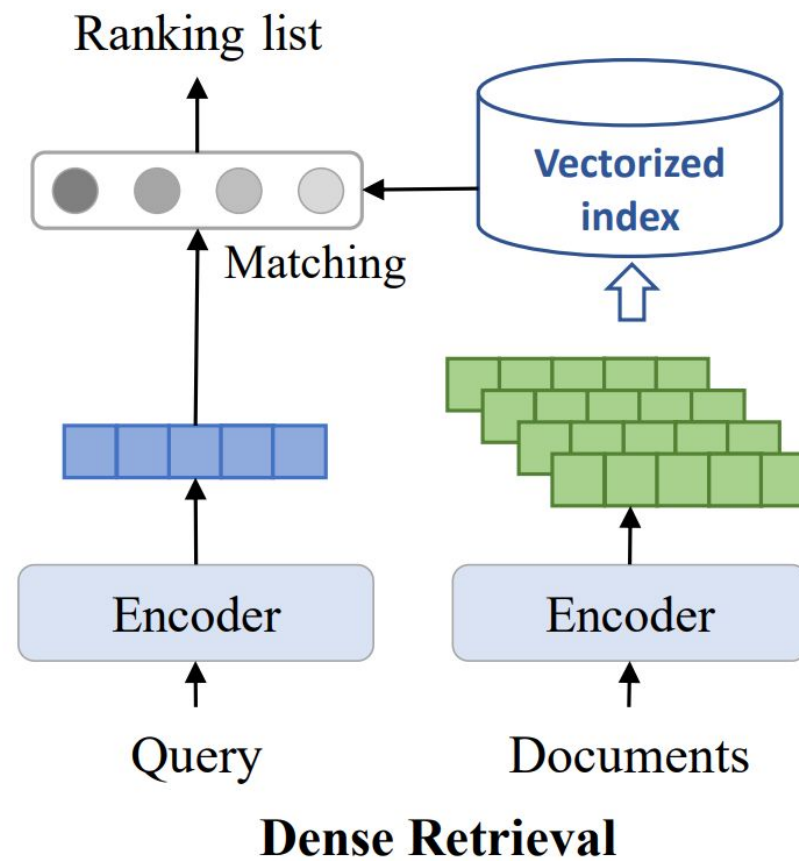
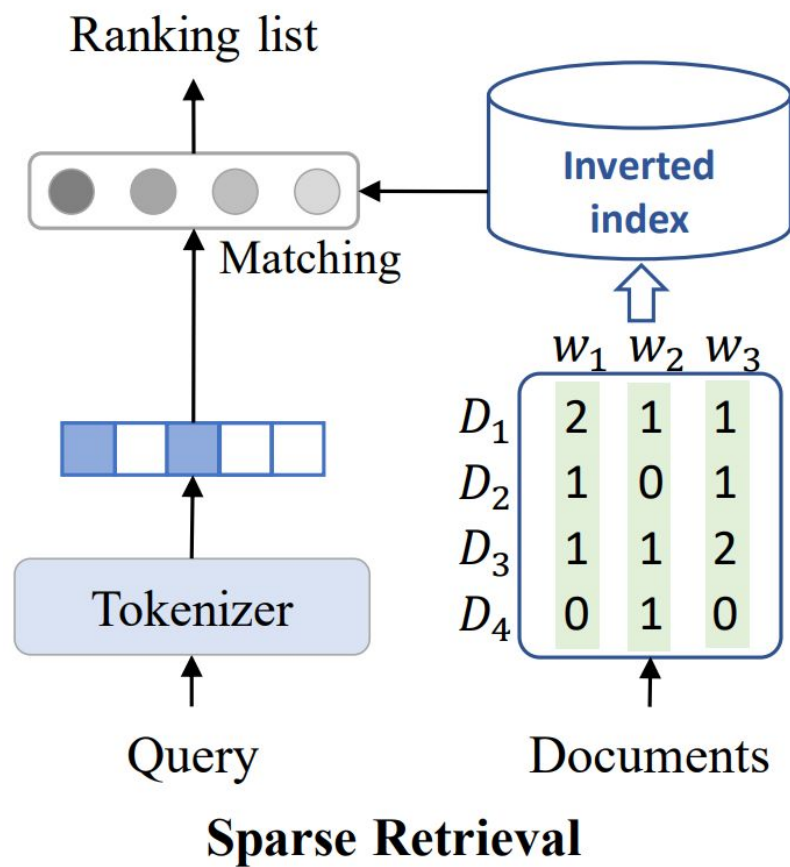
En un sistema real, este proceso puede implementarse mediante:

- **Búsqueda dispersa (sparse retrieval):** Utiliza técnicas tradicionales de **búsqueda** basadas en **palabras clave**, como TF-IDF o BM25.
- **Búsqueda densa (dense retrieval):** Usa **embeddings** semánticos para proyectar la consulta y los documentos en un espacio de alta dimensionalidad y medir la **similitud** entre ellos, normalmente con una métrica como la similitud coseno.

La más potente es la búsqueda densa, pero a veces se puede usar ambos, dando un hybrid search.



Módulo Retriever



Módulo Generator

El **módulo de generación** se encarga de producir una respuesta basada en la **consulta** (query) **q** y los **documentos recuperados** $D=\{d_1,d_2,...,d_n\}$. Este módulo toma la información relevante proporcionada por el módulo de recuperación y genera un texto coherente y preciso.

Pasos:

1. **Entrada inicial:** El modelo recibe como input la consulta **q** y los documentos recuperados **d1,d2,...,dn**.
2. **Generación palabra por palabra:** El modelo genera una secuencia **g** palabra por palabra, condicionada por **q** y los documentos recuperados **di**.
3. **Contextualización:** Durante la generación, el modelo (LLM) considera tanto la información recuperada como la secuencia generada hasta el momento para producir una respuesta relevante.

Conclusiones

- *El lenguaje es inherentemente difícil para las máquinas debido a la **ambigüedad**, dependencia del contexto y pragmática.*
- ***pipeline clásico** modular (tokenización->POS->parsing->modelo) a enfoques **end-to-end** donde una sola red neuronal aprende todas las representaciones intermedias automáticamente*
- *Representación del conocimiento ha evolucionado desde **estructuras formales** (lógica, frames, ontologías) hacia **knowledge graphs** modernos que combinan escalabilidad con razonamiento, siendo fundamentales para motores de búsqueda y sistemas de IA actuales*



Preguntas

gussepppebravo@gmail.com

<https://www.linkedin.com/in/gussepppebravo/>

Links

- [NLP colab](#)
- [RAG colab](#)
- [Intro RAG repo](#)

Tarea (13/09/2025)

- ¿Cómo usarías un embedding para una aplicación?. Por ejemplo, en el RAG, usamos embeddings para representar los chunks como vectores para luego buscar semánticamente.